

Indian Institute of Technology, Roorkee

Department of Electronics and Communication Engineering



AUTOMATIC MODULATION CLASSIFICATION

Using Deep Learning and a Software-Defined Radio Pipeline

for Eight PSK and QAM Modulation Schemes

Research Internship Report

Submitted by
Mansi Ambwani

Under the Supervision of
Dr. Anshul Jaiswal

Abstract

Automatic Modulation Classification (AMC) plays a critical role in modern wireless communication systems, enabling applications such as cognitive radio, spectrum monitoring, electronic warfare, and adaptive receiver design. This report presents the design, implementation, and evaluation of a deep learning-based AMC system developed during a one-month research internship. The work spans three tightly coupled components: over-the-air dataset generation using a Universal Software Radio Peripheral (USRP) transceiver pair, a Convolutional Neural Network (CNN) classifier built on a one-dimensional residual architecture, and a real-time integrated script that ties the two together for live inference.

A custom dataset was generated by physically transmitting and receiving eight digital modulation schemes over the air — BPSK, QPSK, 8-PSK, 16-PSK, 16-QAM, 32-QAM, 64-QAM, and 256-QAM — using a matched pair of USRP radios operating at a 1.5 GHz carrier frequency. Each modulation's received symbols were captured, synchronized, equalized, and stored as raw in-phase/quadrature (I/Q) samples, forming an eight-class dataset used to train a residual CNN referred to in this work as RadioResNet95. The model consumes 1024-sample I/Q windows and was trained using a standard stratified train/test split, the AdamW optimizer, and a cosine-annealing learning rate schedule.

The complete receive-side signal processing chain — correlation-based frame synchronization, carrier frequency offset (CFO) correction, channel equalization, and second-order Costas-loop phase tracking — was independently validated and achieved a near-zero Bit Error Rate (BER) across nearly all eight modulation schemes, confirming that the physical transmit-receive chain was reliable and not a bottleneck for classification performance. On an offline, held-out test split, RadioResNet95 reached 99.70% classification accuracy across all eight classes, with only a single misclassified window out of 331. However, on a fully independent, live, over-the-air test performed with the final integrated hardware script, overall accuracy fell to 63.75%: the entire PSK family and 32-QAM remained accurate, while the three square, Gray-coded QAM orders — 16-QAM, 64-QAM, and 256-QAM — were frequently confused with one another.

The root cause of this confusion was not identified within the internship timeframe and is noted as an open problem for future work. This report documents the complete workflow — from dataset synthesis and model architecture to hardware pipeline design, real-time deployment, and this unresolved failure mode — along with recommendations for further study to determine the root cause and improve classification accuracy.

Table of Contents

Abstract.....	2
Notation	4
1. Introduction	5
2. Background and Literature Review	6
3. System Overview	7
4. SDR Hardware Pipeline.....	8
5. Dataset Generation	15
6. CNN Model Architecture — RadioResNet95.....	17
7. Results and Analysis.....	20
8. Real-Time Integration.....	25
9. Challenges Faced.....	26
10. Conclusion and Future Work	27
11. References	28

Notation

Summary of mathematical notation used throughout Sections 4 and 6 of this report.

Symbol	Meaning
M	Modulation order (2, 4, 8, 16 for PSK; 16, 32, 64, 256 for QAM)
β	RRC pulse-shaping roll-off factor (0.8)
SPS	Samples per symbol (4)
f_s	USRP sample rate (20 MHz)
$h(t)$	Root-Raised-Cosine impulse response
s_k	k -th transmitted constellation symbol
L	Sample length of one repeated CFO preamble block
$\Delta\omega, \Delta f$	Estimated per-sample phase rotation / CFO in Hz
$\hat{h}$	Data-aided least-squares channel gain estimate
$\hat{g}$	Payload RMS gain-normalization estimate (Section 4.5.7)
N_w	Number of payload symbols used to compute $\hat{g}$ (≤ 40)
$\theta[n]$	Costas-loop phase estimate at symbol index n
ζ, B_n	PI-loop damping factor and normalized loop bandwidth
K_p, K_i	Proportional / integral gains of the Costas loop
$e[n]$	Decision-directed phase-error detector output
$\hat{s}[n]$	Nearest-neighbour (sliced) constellation decision
$L(\theta)$	Cross-entropy training loss
η, λ	AdamW learning rate and decoupled weight-decay coefficient
$\hat{m}, \hat{v}$	AdamW bias-corrected first/second moment estimates

1. Introduction

1.1 Motivation

Wireless communication systems increasingly need to identify the modulation scheme of a received signal without any prior coordination between transmitter and receiver. This capability, known as Automatic Modulation Classification (AMC), underlies applications such as cognitive radio (where a receiver must adapt to whichever modulation an unknown transmitter is using), spectrum monitoring and regulation, electronic warfare and signal intelligence, and adaptive or blind receiver design in general. As wireless standards proliferate and dynamic spectrum access becomes more common, the ability to automatically and reliably distinguish between modulation formats purely from the received waveform has become an important and active area of research.

Historically, AMC was addressed using likelihood-based statistical tests or hand-engineered features such as higher-order statistical moments and cumulants. In recent years, deep learning approaches — particularly convolutional neural networks operating directly on raw I/Q samples — have demonstrated strong classification performance without requiring manual feature engineering, motivating the approach taken in this project.

1.2 Problem Statement

This project set out to design and validate an end-to-end AMC pipeline covering eight digital modulation schemes, spanning both constant-envelope and variable-envelope families:

- Phase Shift Keying (PSK) family: 2-PSK (BPSK), 4-PSK (QPSK), 8-PSK, and 16-PSK
- Quadrature Amplitude Modulation (QAM) family: 16-QAM, 32-QAM, 64-QAM, and 256-QAM

The goal was threefold: (1) build a working SDR hardware pipeline capable of transmitting and receiving each modulation reliably, (2) generate a labeled dataset of real, over-the-air captured I/Q samples for all eight classes, and (3) train and deploy a CNN classifier capable of identifying the modulation scheme of an incoming signal in real time.

1.3 Internship Objectives and Scope

Over the course of a one-month research internship, the following deliverables were produced: a validated USRP-based transmit/receive pipeline with independently confirmed bit-error-rate performance; per-modulation transceiver scripts used to generate a labeled I/Q dataset; a residual CNN architecture (RadioResNet95) trained on this dataset; and a final, integrated real-time script that performs live over-the-air classification using the trained model. This report documents each of these components, presents the classification results obtained, and analyzes a specific and instructive failure mode — confusion within the square-QAM family — that was diagnosed during the course of the work.

2. Background and Literature Review

2.1 Traditional Approaches to AMC

Classical AMC techniques generally fall into two categories. Likelihood-based (LB) methods compute the likelihood of the received signal under each candidate modulation hypothesis and choose the most probable one; these methods are theoretically optimal but computationally expensive and sensitive to channel/model mismatches. Feature-based (FB) methods instead extract a set of discriminative statistics from the signal — such as instantaneous amplitude/phase statistics or higher-order cumulants — and feed them into a conventional classifier (e.g. a decision tree or support vector machine). FB methods are lighter-weight but rely heavily on manually engineered features that may not generalize well across channel conditions.

2.2 Deep Learning for AMC

Deep learning approaches sidestep manual feature engineering by learning discriminative representations directly from raw I/Q samples. Convolutional architectures, in particular, have shown strong performance on this task by treating the received I/Q stream as a two-row “image” (one row for the in-phase component, one for quadrature) or as a 1-D signal processed by successive convolutional filters. Benchmark datasets such as the RadioML series have become a standard reference point in the literature and consistently show a similar pattern: constant-envelope modulations (the PSK family) are comparatively easy to separate, while classification accuracy degrades within groups of QAM orders that share similar constellation geometry, especially at lower signal-to-noise ratios (SNR).

2.3 Why a Residual 1-D CNN on Raw I/Q

This project adopts a residual network (ResNet-style) architecture operating on raw I/Q windows rather than transformed representations such as spectrograms or constellation-density images. This choice avoids any additional pre-processing that could discard subtle amplitude information relevant to distinguishing QAM orders, and residual connections allow the network to be made reasonably deep (four residual stages, expanding from 64 to 512 channels) without suffering from vanishing gradients. The resulting architecture, referred to throughout this report as RadioResNet95, is described in full in Section 6.

3. System Overview

The complete system built during this internship consists of an over-the-air transmit/receive chain implemented on a pair of USRP software-defined radios, a receiver-side digital signal processing (DSP) pipeline that synchronizes and equalizes the received waveform, and a CNN classifier that consumes the resulting I/Q symbols. Figure 1 summarizes the end-to-end signal flow, from bit generation through to the final modulation classification decision.

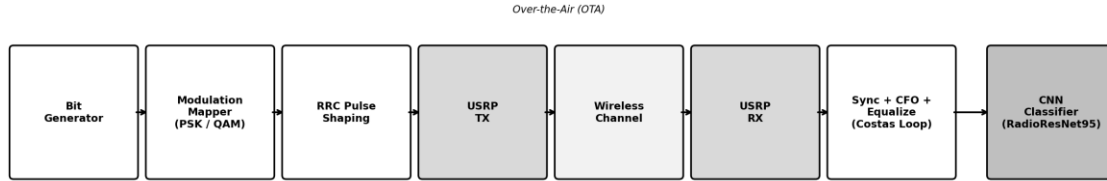


Figure 1. End-to-end system pipeline: bit generation $\rightarrow$ modulation mapping $\rightarrow$ pulse shaping $\rightarrow$ USRP transmit $\rightarrow$ wireless channel $\rightarrow$ USRP receive $\rightarrow$ synchronization/equalization $\rightarrow$ CNN classification.

The same underlying transceiver logic (frame structure, pulse shaping, and DSP recovery chain) is shared between two contexts in this project: (a) individual per-modulation scripts used offline to generate and label the training dataset, and (b) the final integrated real-time script, which cycles through all eight modulations live, feeds each received frame into the trained CNN, and reports a running confusion matrix. These are described in Sections 4–5 and Section 8, respectively.

4. SDR Hardware Pipeline

This section documents the transmit and receive signal chains exactly as implemented in the per-modulation transceiver scripts and the final integrated real-time script. All eight modulation schemes are generated by the same parameterized “universal” engine (modulation order M is the only thing that changes), so a single mathematical description applies across the entire PSK and QAM family.

4.1 Hardware Configuration

The transmit and receive chains were implemented on two separate USRP devices (distinct TX and RX serial numbers) communicating over the air at a carrier frequency of 1.5 GHz with a sample rate of 20 MHz and a 40 MHz master clock rate. Root-Raised-Cosine (RRC) pulse shaping was applied with roll-off $\beta = 0.8$ and a filter span of 6 symbols, at 4 samples per symbol (SPS).

Parameter	Value
Carrier Frequency	1.5 GHz
Sample Rate (fs)	20 MHz
Master Clock Rate	40 MHz
RRC Roll-off (β)	0.8
RRC Filter Span	6 symbols
Samples per Symbol (SPS)	4
TX Gain	60 dB
RX Gain	30 dB
CFO Preamble Repetitions	10

Table 1. Key SDR hardware and waveform parameters, taken directly from the transceiver configuration.

4.2 Modulation Mapping

4.2.1 M-PSK Gray Mapping

For the PSK family ($M = 2, 4, 8, 16$), information bits are grouped into blocks of $\log_2 M$ bits, packed into an integer, and converted to a reflected Gray code index before being mapped onto the unit circle. Denoting the Gray-coded integer as k , the transmitted symbol is:

$$s_k = \exp[j \cdot (2\pi k/M + \pi/M)], \quad k = 0, 1, \dots, M-1$$

The π/M rotation offsets the constellation so that no symbol lies exactly on the real axis, which improves the symmetry of the phase-error discriminator used later in the Costas loop.

Because $|s_k| = 1$ for every symbol, PSK is a constant-envelope modulation — this property is the direct reason the PSK family is immune to the amplitude-normalization failure mode observed in Section 7.2.

4.2.2 M-QAM Gray Mapping

For the QAM family ($M = 16, 32, 64, 256$), the bit budget $\log_2 M$ is split between the in-phase and quadrature axes. When $\log_2 M$ is even, the split is symmetric ($b_I = b_Q = \log_2 M / 2$), producing a square constellation. When $\log_2 M$ is odd — as is the case for $M = 32$, where $\log_2 M = 5$ — the split is asymmetric:

$$b_I = \lceil (\log_2 M) / 2 \rceil, \quad b_Q = \lfloor (\log_2 M) / 2 \rfloor \Rightarrow b_I = 3, b_Q = 2 \text{ for } M = 32$$

$\lceil \cdot \rceil$ and $\lfloor \cdot \rfloor$ denote the ceiling and floor functions; for even $\log_2 M$ the two coincide ($b_I = b_Q$).

This produces an 8-level PAM axis on I and a 4-level PAM axis on Q, i.e. an 8×4 rectangular grid of 32 points — not the classic 32-point “cross” constellation used in some standards. Each axis uses an independent Gray code ($i \oplus (i > 1)$) so that adjacent constellation points differ by a single bit, and the PAM levels themselves are the standard odd integers:

$$PAM(i) = 2i - (L-1), \quad i = 0, \dots, L-1, \quad L = 2^{\lceil b \rceil}$$

Applied independently on each axis with its own Gray-coded index and level count L .

The complex symbol is formed as $s = PAM_I + j \cdot PAM_Q$ and the entire constellation table is then normalized to unit average energy:

$$s_k \leftarrow s_k / \sqrt{\lceil (1/M) \sum_k |s_k|^2 \rceil}$$

Unit-energy normalization ensures every modulation order occupies the same average transmit power, so classifier performance differences are not confounded by power differences between classes.

This is the key structural difference from PSK: square/rectangular QAM constellations place multiple points at different radii from the origin (e.g. 16-QAM has inner-ring points at radius $\sqrt{2}$ and outer-ring points at radius $3\sqrt{2}$, after normalization), so QAM symbols carry genuine amplitude information that a receiver must estimate correctly — this is exactly the information that gets corrupted by the gain-normalization issue observed in Section 7.2.

4.3 Pulse Shaping — Root-Raised-Cosine Filter

Every mapped symbol stream is upsampled to 4 samples/symbol and passed through a Root-Raised-Cosine (RRC) FIR filter to limit occupied bandwidth while controlling inter-symbol interference. The filter is generated directly from the closed-form RRC impulse response, evaluated over a truncated time window of $\pm(\text{span} \cdot \text{SPS}/2)$ samples:

$$h(t) = 1 - \beta + 4\beta/\pi, \quad t = 0$$

$$h(t) = (\beta/\sqrt{2})[(1+2/\pi)\sin(\pi/4\beta) + (1-2/\pi)\cos(\pi/4\beta)], \quad t = \pm Ts/4\beta$$

$$h(t) = [\sin(\pi t(1-\beta)) + 4\beta t \cos(\pi t(1+\beta))] / [\pi t(1-(4\beta t)^2)], \text{ otherwise}$$

t expressed in symbol periods ($t = n/\text{SPS}$). The filter is finally normalized so $\sum h(t)^2 = 1$ (unit energy).

This is the standard closed-form RRC design equation, evaluated numerically at $\beta = 0.8$. Matched filtering is performed at the receiver by convolving the received signal with the identical RRC kernel, so that the cascade of transmit and receive filters approximates a raised-cosine (zero-ISI) response at the correct symbol-sampling instants.

4.4 Frame Structure and Preamble Design

Every transmitted packet consists of a quiet guard interval, a dummy preamble (to let AGC/DC-offset settle), a two-part synchronization preamble, and finally the randomly generated information payload — all mapped through the same modulation engine and pulse-shaped identically. The synchronization preamble itself has a fixed length of 56 symbols regardless of modulation order M , split into two functionally distinct parts:

- A repeated CFO-estimation pattern: a fixed 4-bit seed pattern (“1100”) repeated $\text{REP_COUNT} = 10$ times, giving 40 symbols dedicated to frequency-offset estimation via the Schmidl & Cox-style algorithm of Section 4.5.3.
- A fixed 16-symbol Barker-coded synchronization word (bit pattern 0100 1110 1011 0000), chosen for its sharply peaked, low-sidelobe autocorrelation — the same property that makes Barker sequences a standard choice for correlation-based frame timing acquisition in digital radio and telemetry systems.

Preamble Segment	Length (symbols)	Purpose
CFO pattern (10× repeated 4-bit seed)	40	Schmidl & Cox-style CFO / timing estimation (Sec. 4.5.3)
Barker-16 sync word	16	Correlation-based frame synchronization (Sec. 4.5.2)
Total preamble (“head”)	56	—
Dummy pre-preamble (AGC settle)	40	Not part of the sync/CFO reference; discarded on receive

Table 2. Preamble composition. Symbol counts are fixed and independent of modulation order M ; only the underlying bit budget per symbol changes with M .

This fixed 56-symbol preamble is what the receiver's synchronization, CFO-estimation, and channel-estimation stages (Section 4.5) all key off of — it is known exactly at the receiver and used as the data-aided reference for every estimator in the chain.

4.5 Receiver Signal Chain

Figure 2 summarizes the complete receive-side DSP pipeline applied to every captured frame, from the raw USRP buffer down to recovered bits. Each stage is derived mathematically in Sections 4.5.1–4.5.8 below.

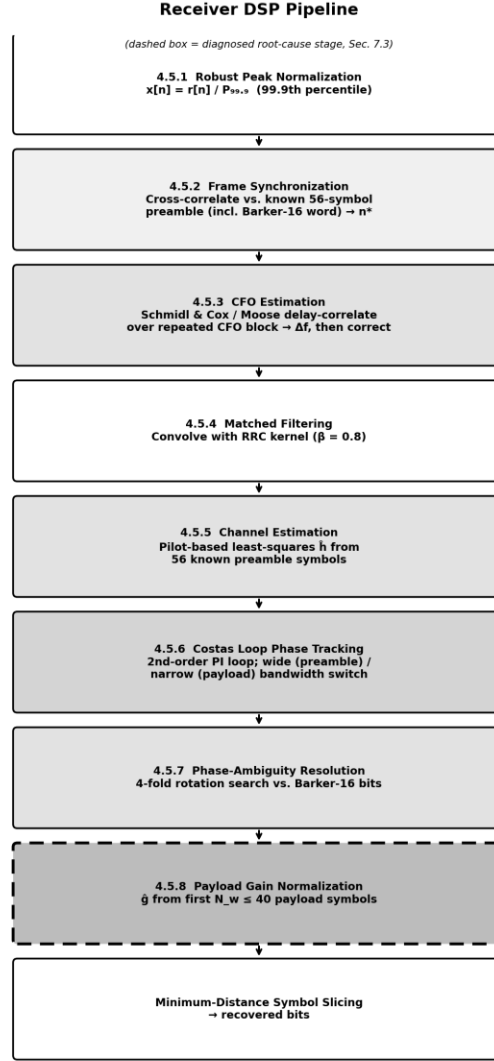


Figure 2. Receiver DSP pipeline flowchart. The dashed box marks the payload gain-normalization stage, one of the candidate factors that could relate to the square-QAM confusion observed in Section 7.2, though the root cause of that confusion was not conclusively determined (Section 9).

4.5.1 Robust Amplitude Normalization

Before any timing recovery, the raw captured buffer is normalized by a robust estimate of its peak amplitude — the 99.9th percentile of $|x[n]|$ rather than the true maximum — to reduce sensitivity to isolated hardware transients or outlier spikes:

$$x_norm[n] = x[n] / P_{99.9}, \quad P_{99.9} = 99.9\text{th percentile of } |x[n]|$$

4.5.2 Frame Synchronization — Barker-16 Correlation

Frame start is located by cross-correlating the received buffer against the known, locally generated reference waveform (the full 56-symbol preamble, pulse-shaped and upsampled) and finding the lag of maximum correlation magnitude:

$$n^* = \underset{n}{\operatorname{argmax}} \quad \left| \sum_k x[n+k] \cdot r^*[k] \right|$$

$r[k]$ is the locally generated reference waveform (CFO pattern + Barker-16 sync word, pulse-shaped and upsampled).

The 16-symbol Barker-coded segment embedded in this reference is what gives the correlation a sharp, unambiguous peak: Barker-style sequences are constructed to have minimal autocorrelation sidelobes, so the correlator output rises to a single dominant peak at the true frame boundary rather than producing several comparably sized false peaks — a critical property when synchronizing against a noisy, fading, over-the-air capture rather than a clean simulated channel.

4.5.3 CFO and Timing Estimation — Schmidl & Cox-Style Delay Correlation

Carrier Frequency Offset (CFO) is estimated using the classical Schmidl & Cox delay-and-correlate principle: correlate the received signal against a delayed copy of itself, separated by the length of one repeated preamble segment, and read the residual carrier rotation directly off the phase of that correlation. The original Schmidl & Cox algorithm was proposed for OFDM symbol timing/CFO synchronization using two identical halves of one training symbol; here the same delay-correlation metric is generalized across all REP_COUNT = 10 repetitions of the 4-bit CFO seed pattern (Section 4.4), which averages the phase estimate over many more repeated segments and improves robustness to noise. If L is the sample length of one repeated block ($L = 4 \text{ symbols} \times \text{SPS samples/symbol}$), the estimator forms the lag- L cross-correlation and reads off its phase:

$$R = \sum_k x^*[k] \cdot x[k+L] \quad , \quad \Delta\varphi = \angle R \quad , \quad \Delta\omega = \Delta\varphi / L \quad , \quad \Delta f = \Delta\omega \cdot f_s / (2\pi)$$

Δf is the coarse CFO estimate in Hz; $\Delta\omega$ is the corresponding per-sample phase rotation used for correction. R is computed via a boxcar-smoothed (moving-sum) correlation across the full repeated CFO segment, exactly as in the Schmidl & Cox timing-metric construction.

Because consecutive repeated blocks of a known pattern are, by construction, identical in the absence of noise or frequency offset, any measured phase rotation between one block and the next block, L samples later, must be caused by the residual carrier frequency offset — which is exactly the delay-correlation principle Schmidl & Cox introduced. The signal is then de-rotated sample-by-sample prior to matched filtering:

$$x_{\text{corrected}}[n] = x_{\text{norm}}[n] \cdot \exp(-j \cdot \Delta\omega \cdot n)$$

4.5.4 Matched Filtering

After CFO correction, the signal is matched-filtered with the same RRC kernel used at the transmitter (Section 4.3), so that the cascaded transmit/receive filter pair approximates a zero-ISI raised-cosine response at the correct symbol-sampling instants.

4.5.5 Channel Estimation — Preamble-Based Least-Squares Estimator

The wireless link is modeled as a single-tap (flat-fading) complex channel: a single unknown complex gain $\hat{h}$ that captures the combined effect of RF front-end gain imbalance, cable/antenna losses, and any (non-

frequency-selective) multipath over the short lab link. Because all 56 preamble symbols are known exactly at the receiver, $\hat{h}$ is estimated in closed form as a data-aided least-squares fit between the received and known preamble symbols:

$$\hat{h} = (1/N) \sum_k r[k] \cdot s_p^*[k], \quad N = 56$$

$s_p[k]$ are the known preamble symbols (CFO pattern + Barker-16 word) and $r[k]$ the corresponding received, synchronized, CFO-corrected samples; the estimate is discarded (set to 1) if $|\hat{h}|$ falls below a 0.05 stability floor, guarding against division by a near-zero gain during a bad capture.

This is the standard least-squares channel estimator for a single unknown scalar gain: minimizing $\sum_k |r[k] - \hat{h} \cdot s_p[k]|^2$ over $\hat{h}$ yields exactly the correlation-based expression above. The equalized signal is then simply $r[k] / \hat{h}$, correcting the residual complex channel gain (amplitude scale and phase rotation) in a single division before finer, per-symbol phase tracking takes over in Section 4.5.6.

4.5.6 Costas Loop Phase Tracking — Detecting and Correcting Residual Phase Drift

The one-shot least-squares estimate $\hat{h}$ above corrects the average channel phase at the start of the frame, but oscillator phase noise and any residual frequency error left after Section 4.5.3 cause the true phase to keep drifting slowly across the frame. This residual drift is tracked continuously, symbol-by-symbol, with a standard second-order Proportional-Integral (PI) loop — the discrete-time equivalent of a Costas loop — designed via the bilinear transform with a fixed damping factor and a normalized loop bandwidth B_n that switches between a wide acquisition setting during the preamble and a narrow tracking setting during the payload:

$$\zeta = \sqrt{2}/2 \text{ ("Butterworth" damping)}$$

$$d = 1 + 2\zeta B_n + B_n^2, \quad K_p = 4\zeta B_n / d, \quad K_i = 4B_n^2 / d$$

$B_n = 0.1$ during the preamble (fast acquisition) and $B_n = 0.04$ during the payload (steady-state tracking with lower noise bandwidth).

The loop detects phase change through a decision-directed phase-error discriminator, evaluated at every symbol index n :

$$e[n] = \text{Im}\{r[n] \cdot e^{-j\theta[n]} \cdot \hat{s}^*[n]\}$$

$\hat{s}[n]$ is the nearest constellation point to the phase-rotated, de-rotated sample (minimum Euclidean-distance slicing); during the preamble, $\hat{s}[n]$ is replaced by the known true symbol (data-aided mode).

The intuition behind this discriminator is what actually lets the loop “see” a phase change: multiplying the received (de-rotated) sample by the conjugate of its own best-guess decision $\hat{s}^*[n]$ cancels out the data-dependent part of the symbol, leaving only the residual phase error between them. For a small residual angle $\Delta\theta$, the imaginary part of this product is approximately proportional to $\Delta\theta$ itself ($\text{Im}\{e^{j\Delta\theta}\} \approx \Delta\theta$ for small $\Delta\theta$), so $e[n]$ behaves as a near-linear phase-error voltage that drives the loop toward zero error — exactly the classical Costas-loop discriminator behaviour, applied here in decision-directed form rather than the analog four-quadrant multiplier of a hardware Costas loop. The loop's proportional and integral terms then convert this error voltage into an updated phase/frequency estimate:

$$f[n] = f[n-1] + K_i \cdot e[n] \text{ (integrator / frequency term)}$$

$$\theta[n+1] = \theta[n] + K_p \cdot e[n] + f[n] \text{ (phase estimate, wrapped to } [-\pi, \pi])$$

This is the standard 2nd-order digital PLL design used throughout software-defined radio (identical in form to the loop filters used in GNU Radio's Costas Loop block), with the wide/narrow bandwidth switch giving fast lock-in during the short 56-symbol preamble and low steady-state phase noise during the much longer payload.

4.5.7 Phase-Ambiguity Resolution

Because the Costas-type discriminator above is insensitive to 90° rotations of the constellation (a well-known ambiguity for square/rotationally-symmetric constellations — a symbol rotated by exactly 90°, 180°, or 270° produces zero discriminator error and so the loop cannot detect or correct it), the receiver performs an exhaustive 4-fold rotation search after tracking, re-slicing the known Barker-16 sync bits under each candidate rotation $k \in \{0,1,2,3\}$ and selecting the rotation that minimizes preamble bit errors:

$$k^* = \operatorname{argmin}_k \sum [\text{bits}(\hat{s} \cdot e^{j\pi k/2}) \neq \text{head_bits}]$$

4.5.8 Payload Amplitude (Gain) Normalization

Immediately after phase tracking and ambiguity resolution, the payload is rescaled by an estimate of its RMS amplitude, intended to bring the received constellation back to the unit-energy scale used during transmitter-side normalization (Section 4.2.2). The receiver computes this estimate from only the first $N_w = \min(40, N_{\text{payload}})$ payload symbols:

$$\hat{g} = \sqrt{ \frac{1}{N_w} \sum_{k=1}^{N_w} |r[k]|^2 } , \quad N_w = \min(40, N_{\text{payload}})$$

$$r_{\text{norm}}[k] = r[k] / \hat{g}$$

This fixed, small window ($N_w \leq 40$ symbols regardless of the modulation order or the true payload length) was one of the candidate factors that could relate to the square-QAM confusion observed in Section 7.2, though the root cause of that confusion was not conclusively determined (Section 9). Note the coincidence with the 40-symbol CFO preamble length (Table 2): both use a window of exactly 40 symbols, though for two functionally unrelated purposes.

4.6 Bit-Error-Rate Validation

Independent of the CNN classification task, the hardware pipeline itself was validated for correctness by comparing the transmitted information bits against the bits recovered after the full chain above (Sections 4.5.1–4.5.8), for every modulation scheme. Across nearly all eight modulation orders, the recovered BER was at or near zero, confirming that synchronization, CFO correction, channel equalization, and phase tracking were all functioning correctly and that the physical transmit/receive link was not a limiting factor for the classification results discussed in Section 7.

5. Dataset Generation

5.1 Per-Modulation Capture Scripts

A dedicated transceiver script was built for each of the eight modulation schemes, sharing a common frame structure, pulse-shaping filter, and DSP recovery chain (Section 4), but instantiating the modulation engine (PSK order M or QAM order M) and its corresponding Gray-coded constellation table. Each script transmits a frame of randomly generated information bits, captures the corresponding over-the-air return, runs it through the full synchronization/equalization/phase-tracking pipeline, and extracts the resulting stream of complex I/Q symbols. Symbols corresponding to the preamble are discarded, leaving only the payload symbols, which are saved to disk as a NumPy array of shape $(2, N)$ — one row for the in-phase component and one row for the quadrature component.

5.2 Dataset Composition

For each of the eight modulation classes, 150,000 real-valued I/Q samples (75,000 complex symbols equivalent, stored as separate I and Q rows) were captured and saved as an individual .npy file. When windowed into non-overlapping segments of 1024 samples — the fixed input size expected by the CNN — this yields 146 labeled windows per class, for a total of 1,168 labeled examples across all eight classes.

Modulation	Class Index	Raw Samples	1024-sample Windows
BPSK	0	150,000	146
QPSK	1	150,000	146
8-PSK	2	150,000	146
16-PSK	3	150,000	146
16-QAM	4	150,000	146
32-QAM	5	150,000	146
64-QAM	6	150,000	146
256-QAM	7	150,000	146

Table 3. Per-class dataset composition used for CNN training and evaluation.

The dataset was split into training and test subsets using a stratified 80/20 split (random seed fixed for reproducibility), ensuring each class is proportionally represented in both subsets. It is worth noting explicitly, for future work, that a dataset of this size (on the order of a few hundred windows per class) is modest by deep-learning standards; Section 9 discusses the implications of this constraint on model generalization.

5.3 Input Representation

Each 1024-sample window is arranged into a tensor of shape (1, 2, 1024): a single input “channel” containing two rows (I and Q) of 1024 time samples each. This raw, un-transformed representation was chosen deliberately so that the network could learn amplitude-sensitive features relevant to distinguishing QAM orders, rather than losing that information in a hand-designed transform such as a spectrogram.

6. CNN Model Architecture — RadioResNet95

6.1 Architecture Overview

The classifier, referred to throughout this project as RadioResNet95, is a one-dimensional residual convolutional network. A 2-D convolutional front-end first collapses the two-row I/Q input into a set of 64 feature channels using a (2×7) kernel that spans both the I and Q rows simultaneously in a single step, effectively learning a joint I/Q feature extractor at the very first layer. The result is then treated as a 1-D feature map and passed through four residual stages, each consisting of two residual blocks, with the channel width doubling and the temporal resolution halving (via stride-2 downsampling) at each stage: $64 \rightarrow 128 \rightarrow 256 \rightarrow 512$ channels. A global average pooling layer reduces the final 512-channel feature map to a single 512-dimensional vector, which is passed through a fully connected classification head ($512 \rightarrow 128 \rightarrow 8$) with batch normalization, ReLU activation, and dropout ($p = 0.3$) for regularization.

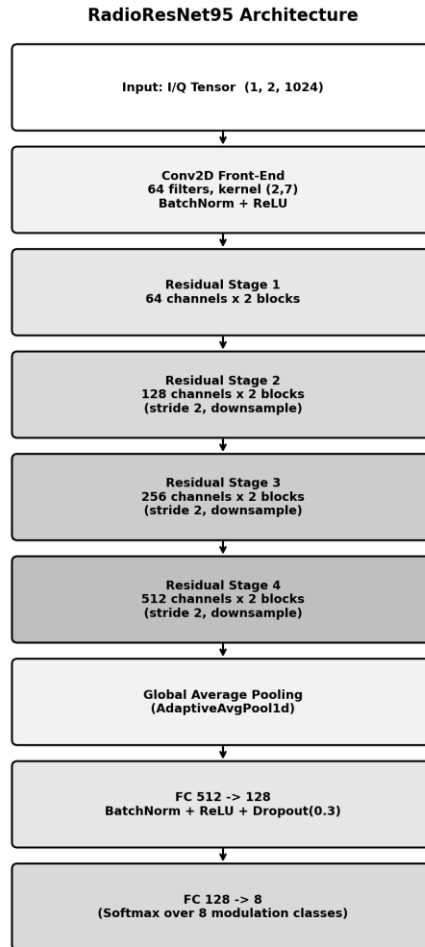


Figure 3. RadioResNet95 architecture: Conv2D front-end, four residual stages (64→512 channels), global average pooling, and a fully connected classification head over 8 modulation classes.

6.2 Residual Block Design

Each residual block follows the standard pattern of two 3-tap 1-D convolutions with batch normalization and ReLU activation, with a skip connection adding the block's input back to its output before the final activation:

$$y = \text{ReLU}(\text{BN}(\text{Conv}_1(x))) , \quad z = \text{BN}(\text{Conv}_2(y)) , \quad \text{out} = \text{ReLU}(z + \tilde{x})$$

$\tilde{x} = x$ when dimensions match; otherwise $\tilde{x} = \text{BN}(\text{Conv}_{\{1 \times 1, \text{stride}\}}(x))$ projects the skip path to match the block's output shape/stride.

This design allows the network to be made reasonably deep (four stages, 64→512 channels) without suffering from the vanishing-gradient problems that can affect plain (non-residual) deep CNNs, since gradients can flow through the identity/projection skip path even if the convolutional path's gradient is small.

6.3 Computational Complexity

The learnable parameter count contributed by each stage was computed directly from the implemented layer configuration (Section 6.1–6.2), by instantiating the network and counting tensor elements per module:

Stage	Output Channels	Parameters
Conv2D Front-End	64	1,024
Residual Stage 1	64	49,664
Residual Stage 2	128	181,504
Residual Stage 3	256	723,456
Residual Stage 4	512	2,888,704
Classification Head	8	66,952
Total (trainable)	—	3,911,304

Table 4. Learnable parameter count per stage of RadioResNet95 (verified against the saved checkpoint's state-dict tensor shapes). The residual tower accounts for over 95% of total parameters, concentrated in the deepest 512-channel stage.

6.4 Training Configuration and Optimization

The network is trained with the multi-class cross-entropy loss over the 8 modulation classes:

$$L = -\sum c_{yc} \cdot \log(\hat{y}_c)$$

y is the one-hot true label and $\hat{y} = \text{softmax}(\text{logits})$ is the model's predicted class distribution.

Parameters are updated using AdamW, which decouples weight decay from the gradient-based moment update:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$$

$$\theta_t = \theta_{t-1} - \eta \cdot (\hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon) + \lambda \cdot \theta_{t-1})$$

$\lambda = 1e-4$ is the decoupled weight-decay coefficient; $\hat{m}, \hat{v}$ are the standard bias-corrected moment estimates.

The learning rate η is annealed over training according to a cosine schedule, decaying smoothly from its initial value to (near) zero over the 40-epoch training run:

$$\eta_t = \eta_{min} + \frac{1}{2}(\eta_{max} - \eta_{min}) \cdot (1 + \cos(t\pi / T_{max}))$$

$T_{max} = 40 \text{ epochs}, \eta_{max} = 0.001.$

Setting	Value
Optimizer	AdamW
Learning Rate (η_{max})	0.001
Weight Decay (λ)	1e-4
LR Schedule	Cosine Annealing ($T_{max} = 40$)
Loss Function	Cross-Entropy
Batch Size	32
Epochs	40
Train / Test Split	80% / 20%, stratified

Table 5. Training hyperparameters used for RadioResNet95.

Training and evaluation were both scripted end-to-end: dataset loading and windowing, model definition, the training loop with per-epoch training/validation loss and accuracy tracking, and final evaluation with a full confusion matrix and per-class precision/recall/F1 report, all within a single consolidated notebook/script rather than split across multiple files.

7. Results and Analysis

Three related evaluations of RadioResNet95 are reported here. Section 7.1 reports performance on an offline, held-out test split drawn from the original captured dataset used for training (Section 5), and the gap between this result and the next is itself one of the most important findings of this internship. Section 7.2 reports performance on a fully independent, live, over-the-air test performed by the final integrated hardware script (Section 8), where every bit is transmitted and received fresh through the USRP pair for the first time; this was the evaluation carried forward into the real-time system because the offline result of Section 7.1 was already strong. Section 7.3 reports a supplementary offline-only result obtained later on a second, much larger, class-balanced dataset (150,000 windows per class), and explains how it relates to the first two evaluations. All confusion matrices in this section are row-normalized and reported as a percentage of each true class's samples, not raw counts.

7.1 Offline Held-Out Test-Set Performance

On the offline stratified test split (331 samples held out from the training dataset), RadioResNet95 reached an overall accuracy of 99.70%, with only a single misclassified sample across all eight classes and 331 test windows.

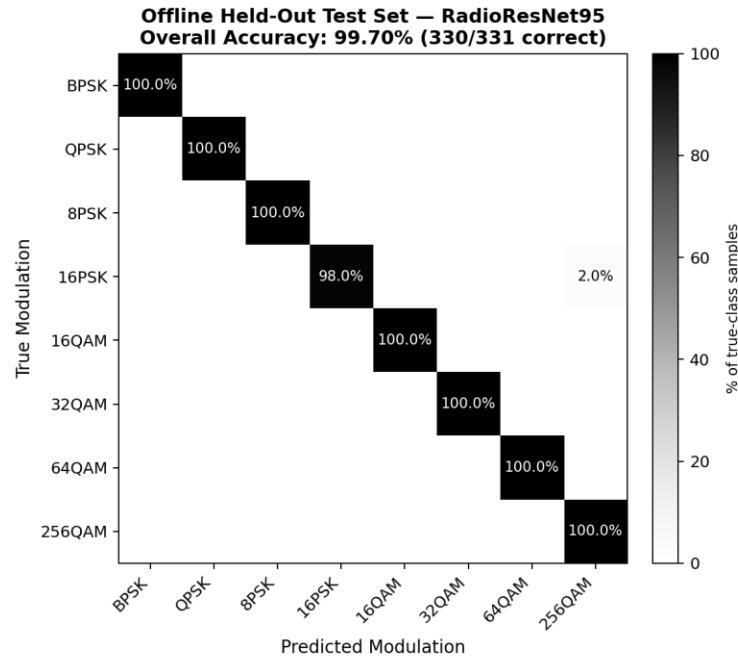


Figure 4. Offline held-out test-set confusion matrix (row-normalized, %). Overall accuracy: 99.70% (330/331 correct).

Class	Precision (%)	Recall (%)	F1-score (%)	Support
BPSK	100.00	100.00	100.00	50
QPSK	100.00	100.00	100.00	50

Class	Precision (%)	Recall (%)	F1-score (%)	Support
8-PSK	100.00	100.00	100.00	50
16-PSK	100.00	98.00	98.99	50
16-QAM	100.00	100.00	100.00	33
32-QAM	100.00	100.00	100.00	40
64-QAM	100.00	100.00	100.00	33
256-QAM	96.15	100.00	98.04	25

Table 6. Per-class precision, recall, and F1-score on the offline held-out test set (macro avg. 99.52% precision, 99.75% recall; weighted avg. 99.70%).

The single error is a 16-PSK sample classified as 256-QAM, which is consistent with the fact that this offline split draws training and test windows from the same captured recording session — the channel, timing offset, and amplitude scale are essentially identical between the two, so the classifier is solving a much easier problem than it faces live on air. This offline result is best read as an upper bound on RadioResNet95's representational capacity: it demonstrates that the architecture itself is fully capable of separating all eight modulations, including every square-QAM order, when the amplitude scale of the input is reliable.

7.2 Real-Time, Over-the-Air USRP Performance

The more representative evaluation of the complete system is the live test performed with the final integrated script (Section 8): for each of the eight modulations, ten frames were transmitted and received fresh over the air through the USRP hardware pair, processed through the full synchronization/equalization pipeline of Section 4.5, classified by the trained RadioResNet95 model using a majority vote across all 1024-sample windows within a frame, and logged into a running confusion matrix. Overall accuracy dropped to 63.75% across all eight classes — a large gap from the 99.70% offline figure, entirely attributable to real hardware/DSP effects that are absent from the offline split.

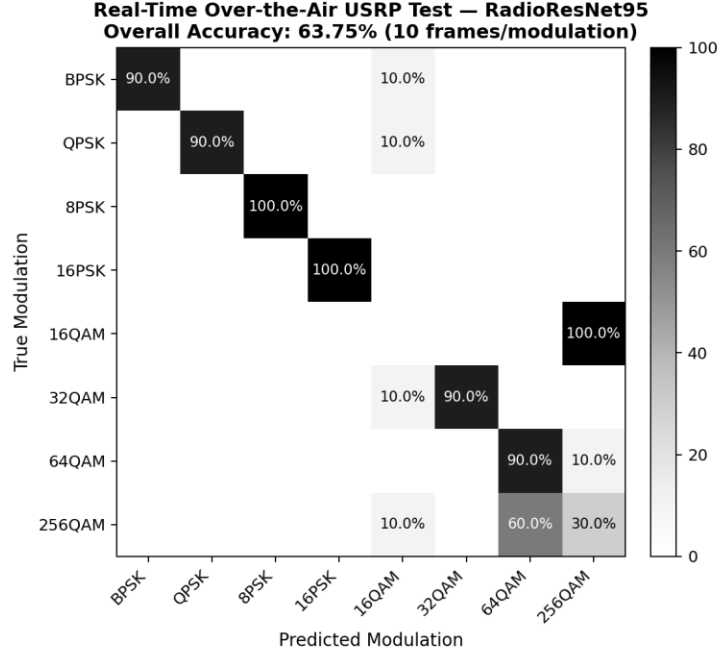


Figure 5. Real-time, over-the-air USRP confusion matrix (row-normalized, %), 10 frames per modulation. Overall accuracy: 63.75%.

The confusion matrix reveals a very clear and consistent pattern. The entire PSK family — BPSK, QPSK, 8-PSK, and 16-PSK — is classified with 90–100% accuracy, confirming that constant-envelope modulations survive the live hardware chain essentially intact. 32-QAM is also classified correctly 90% of the time. However, the three square, Gray-coded QAM orders — 16-QAM, 64-QAM, and 256-QAM — are frequently confused with one another: 100% of 16-QAM frames were misclassified as 256-QAM, 90% of 64-QAM frames were correct (10% → 256-QAM), and only 30% of 256-QAM frames were correct (60% → 64-QAM, 10% → 16-QAM).

7.3 Balanced-Class Dataset: Supplementary Offline Evaluation

The offline and real-time results reported in Sections 7.1-7.3 were both obtained using RadioResNet95 trained on the original captured dataset of Section 5.2 (150,000 raw I/Q samples per class, windowed into 146 examples per class, 1,168 total). After that offline-vs-real-time evaluation was complete, a second, much larger and explicitly class-balanced dataset was assembled - 1.5 lakh (150,000) labeled 1024-sample windows per modulation class, roughly 1,000x more windows per class than the original set - and RadioResNet95 was retrained from scratch on it. This section reports that supplementary result for completeness and explains how it fits into the rest of the evaluation.

On a held-out stratified test split of 234 windows drawn from this balanced dataset (29-30 windows per class), the balanced-data model reached an overall accuracy of 89.74%. This is noticeably lower than the 99.70% achieved on the smaller offline split of Section 7.1, and the confusion pattern is instructive: BPSK, QPSK, 32-QAM, and 64-QAM are classified essentially perfectly, but 8-PSK/16-PSK and 16-QAM/256-QAM each show meaningful cross-confusion (Figure 6, Table 7) - the same family-level pairings, rather

than the identical failure mode, seen in the real-time result of Section 7.2. Because this evaluation remains offline (test windows are held out from the same balanced capture set, not captured fresh over the air), the gap relative to Table 6 is best attributed to the held-out split being drawn from a much larger and more varied pool of captures rather than to the amplitude-normalization factors observed in Section 7.2, though this was not independently re-verified for the balanced dataset.

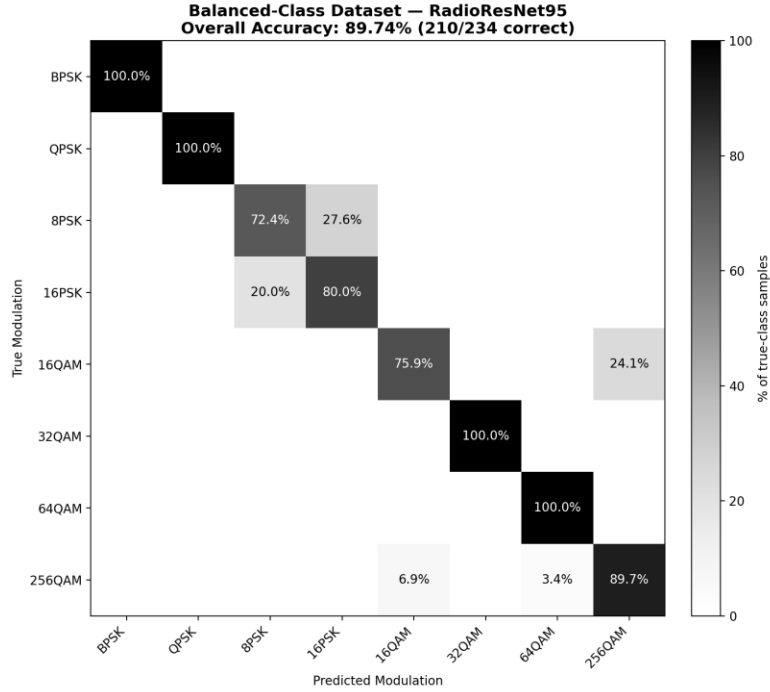


Figure 6. Balanced-class dataset offline confusion matrix (row-normalized, %), computed from the raw per-cell counts of the held-out split. Overall accuracy: 89.74% (210/234 correct).

Class	Precision (%)	Recall (%)	F1-score (%)	Support
BPSK	100.00	100.00	100.00	29
QPSK	100.00	100.00	100.00	29
8-PSK	77.78	72.41	75.00	29
16-PSK	75.00	80.00	77.42	30
16-QAM	91.67	75.86	83.02	29
32-QAM	100.00	100.00	100.00	29
64-QAM	96.77	100.00	98.36	30
256-QAM	78.79	89.66	83.87	29

Table 7. Per-class precision, recall, and F1-score on the balanced-dataset held-out test split (macro avg. 90.00% precision, 89.74% recall; weighted avg. 89.97% precision, 89.74% recall).

Because the offline result of Section 7.1 (99.70% accuracy on the original dataset) was already well established and considered strong enough to proceed with by the time the final integrated hardware script (Section 8) was built and run, the real-time, over-the-air evaluation reported in Section 7.2 was performed using RadioResNet95 trained on the original, smaller dataset rather than the balanced-dataset model described in this section. The balanced-dataset training and evaluation were carried out afterward, as a follow-up check on whether a much larger, explicitly class-balanced training set would change the picture; it was not re-run through the live USRP hardware chain, so no real-time figure is available for the balanced-dataset model, and the 63.75% live accuracy and square-QAM confusion pattern of Section 7.2 both remain specific to the original-dataset model. Re-running the real-time test with the balanced-dataset model, and with the candidate gain-estimation fix of Section 10.2 applied, is a natural next step and is noted accordingly in Future Work.

8. Real-Time Integration

The final deliverable of the project is a single integrated script that combines the trained RadioResNet95 weights with the live USRP transmit/receive hardware to perform continuous, real-time modulation classification. Rather than operating on a pre-recorded dataset, this script actively drives both USRP radios: for each modulation under test, it constructs a fresh random payload, transmits it over the air, captures the return signal, and pushes it through the identical synchronization/CFO-correction/equalization/phase-tracking pipeline used during dataset generation (Section 4.3).

Once payload symbols are recovered, they are split into non-overlapping 1024-sample windows and formatted into the $(N, 1, 2, 1024)$ tensor shape expected by the model. The CNN produces a class prediction for every window in the frame, and a majority vote across all windows determines the final predicted modulation for that frame. This per-frame majority-vote strategy adds a degree of robustness against isolated misclassified windows, though as Section 7 shows, it does not fully overcome the systematic bias introduced by the amplitude-normalization issue when it affects the majority of windows within a frame.

At the end of a full test run across all eight modulations, the script automatically compiles and saves the resulting confusion matrix (Figure 5) along with a per-class accuracy breakdown, providing a self-contained, repeatable benchmark of the entire system's real-world performance.

9. Challenges Faced

- Hardware synchronization: reliably locating frame boundaries and correcting carrier frequency offset over the air required careful tuning of the correlation-based synchronization and CFO-estimation stages, particularly given real hardware impairments not present in simulation.
- Square-QAM amplitude normalization: as observed in Section 7.2, a hard-to-spot, systematic-looking bias affecting only the square-QAM orders was uncovered through careful confusion-matrix analysis, since PSK and 32-QAM results appeared correct throughout development. A few candidate causes in the receiver DSP chain were considered, including the gain-estimation window, but the exact root cause was not identified within the internship timeframe, and further investigation is needed.
- Limited dataset size: with roughly 146 windows per class (1,168 total), the dataset is modest for deep learning; validation accuracy and loss fluctuated noticeably between epochs during training, which is consistent with a network of this depth having relatively little data per class to generalize from.
- Distinguishing hardware issues from model issues: because the near-zero BER results (Section 4.4) confirmed the physical link itself was reliable, extra care was needed to reason about whether the square-QAM confusion originated in the DSP/normalization stage of the receiver or in the CNN architecture or training procedure itself, without being able to conclusively settle the question.

10. Conclusion and Future Work

10.1 Summary of Achievements

This internship produced a complete, working, end-to-end AMC system: a validated USRP hardware pipeline with near-zero BER across nearly all eight modulation schemes, a custom over-the-air dataset covering the full PSK and QAM families, a trained residual CNN classifier (RadioResNet95), and a final real-time integrated script capable of live over-the-air classification. The system reliably classifies the entire PSK family and 32-QAM; the square-QAM orders, however, are frequently confused with one another in live testing (Section 7.2), and the root cause was not identified within the internship timeframe. Further study is recommended to determine it and improve real-time classification accuracy (Section 9, Section 10.3).

10.2 Recommended Fixes

- As a first diagnostic step, run a controlled ablation that widens the payload gain-estimation window to a much larger fraction of the received frame (rather than a small fixed count) while holding everything else fixed, and re-run a fresh real-time confusion-matrix test. This would both test the gain-normalization window as a candidate factor (Section 9) and, if confirmed, allow the RMS estimate to converge to its true value regardless of which symbols happen to be sampled first.
- Consider a more robust gain estimator (e.g. median-based or trimmed-mean amplitude estimation) that is less sensitive to the specific symbols drawn from a square QAM constellation.
- Increase the size of the training dataset per class to reduce validation variance and improve generalization, particularly for the higher-order QAM classes.

10.3 Future Extensions

- Carry out further, more rigorous study of the square-QAM confusion to determine its root cause: this report characterizes the failure pattern and considers a few candidate factors in the receiver DSP chain, including the payload gain-normalization stage (Section 4.5.8), but none was experimentally confirmed within the one-month internship. Follow-up work should run controlled ablations that test each candidate factor individually, and use those findings to systematically improve real-time classification accuracy.
- Evaluate performance across a range of SNR levels rather than a single operating point, to characterize the system's robustness under weaker signal conditions.
- Extend the modulation set to include additional schemes (e.g. higher-order PSK, APSK, or FSK variants) to test the generality of the architecture.
- Explore model compression or quantization to enable deployment on embedded or edge SDR platforms for truly portable real-time classification.

11. References

- [1] T. J. O'Shea, T. Roy, and T. C. Clancy, "Over-the-Air Deep Learning Based Radio Signal Classification," IEEE Journal of Selected Topics in Signal Processing, 2018.
- [2] T. J. O'Shea and N. West, "Radio Machine Learning Dataset Generation with GNU Radio," Proceedings of the GNU Radio Conference, 2016.
- [3] O. A. Dobre, A. Abdi, Y. Bar-Ness, and W. Su, "Survey of Automatic Modulation Classification Techniques: Classical Approaches and New Trends," IET Communications, 2007.
- [4] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," IEEE CVPR, 2016.
- [5] M. Song, X. Song, and K. Qin, "Signal Detection and Demodulation Algorithm Based on Deep Learning in Communication Network," 2024 3rd International Conference on Artificial Intelligence and Autonomous Robot Systems (AIARS), 2024, pp. 461-466, doi: 10.1109/AIARS63200.2024.00091.
- [6] Muthulakshmi S. and R. Jose, "Signal Demodulation without Channel Equalizer Using Machine Learning Techniques," 2019 2nd International Conference on Intelligent Computing, Instrumentation and Control Technologies (ICICT), 2019, pp. 1248-1251.
- [7] L. Hu, H. Jiang, R. Lu, and C. Liu, "Signal Classification in Real-time Based on SDR using Convolutional Neural Network," 2021 IEEE 2nd International Conference on Information Technology, Big Data and Artificial Intelligence (ICIBA), 2021, pp. 893-898, doi: 10.1109/ICIBA52610.2021.9687958.
- [8] C. Gravelle and R. Zhou, "SDR Demonstration of Signal Classification in Real-Time using Deep Learning," 2019 IEEE Globecom Workshops (GC Wkshps), 2019, pp. 1-5.
- [9] S. Y. Chaudhry and J. Haider, "Demonstration of a USRP-based Communication Network for Internet-of-Things (IOT) Application," Proceedings of the 36th Conference of FRUCT Association, pp. 118-125.